How to verify the results of the paper

E. Girondo and C. Reyes, *Extremal disc packings in compact hyperbolic surfaces of minimal genus*

This guide is for readers who want to check, by themselves, what the paper says about one surface or about all of them. It assumes no knowledge of the programs, only of the paper. Every command below has been run, exactly as written, on a fresh copy of this folder; the outputs quoted are the ones you should obtain. A formatted copy of this guide is included as `VERIFYING.pdf`; this `.md` file is the reference version.

Time needed: about 15 minutes to install the software and check one surface; about 45 minutes to recompute everything from scratch.

---

1. What can be checked, and what has to be trusted

The results of Section 6 of the paper rest on three kinds of computations, which are checked in different ways.

(a) The list of surfaces. Each surface is D/K for a torsion-free subgroup K of a triangle group, and the list of these subgroups up to conjugation is computed by GAP (`LowIndexSubgroupsFpGroup`). Here you trust GAP, a standard and widely used system. The numbers of classes can be compared with Table 2 of the paper (Section 5 below, first line of the table).

(b) The existence of every packing that the paper lists. Every extremal packing is confirmed by an exact algebraic identity (Proposition 4.1): a matrix α with entries in the number field L_N such that, for each side pairing s of the fundamental domain of K, we have s·α = ±α·w_s with w_s an element of the triangle group. The matrices α and the words w_s are stored in the data files, and these identities are checked by `verify_packings.py`, a script of about 200 lines that does **not** use the search programs at all: it only needs exact arithmetic in L_N (PARI) and the permutation representation produced by GAP. So, to be sure that the packings exist, you do not need to trust the programs that found them, only this short script, which is written to be read (its first 30 lines explain what it checks), together with PARI and GAP: a packing that passes this check exists, whatever the search did. This is the quickest and most convincing check, and it takes about one minute for all the surfaces (Section 3).

(c) The fact that there are no other packings. This is the part that relies on the search described in Sections 3 and 4 of the paper: a complete list of admissible distances, the intersections of the ED-cycles (equi-displaced cycles) of two side pairings inside the circumscribed disc, and the rejection of every other candidate point. It cannot be reduced to an identity that can be checked independently. It can be checked in four ways:

- by running the search again, for one surface (a few seconds, Section 4) or for all of them (Section 6); the computation is deterministic and gives exactly the same output every time (it has been repeated on different computers, with different versions of GAP);

- by following the search of any particular surface step by step: `check_surface.py --details` lists every admissible distance, every intersection point of the ED-cycles and what happens to it, and `draw_surface.py --edcycles` draws them (Section 4.3); this is what Figure 2 of the paper shows for X_0;
- by reading the code: Section 9 says which function implements each step of the algorithm of the paper. The search uses floating point numbers (40 significant digits) only to decide whether a displacement is an admissible distance, and every such decision is taken with a wide margin; `margins.py` measures these margins (Section 5);
- through the independent checks that the results pass: the algorithm reproduces the classifications obtained by other methods (Section 7); the isometries found satisfy the Euler characteristic relation for the quotient by the isometry group on all the surfaces, and all the Dirichlet domains have the right area and vertex cycles (Sections 4.4 to 4.6).

The remaining statements (automorphism groups, which packings are related by isometries, which classes give the same surface, Weierstrass points) are computed from the stored matrices α and words w_s, and each one is reproduced by a short script listed in Section 5.

A few statements are proved by hand in the paper from computed data and are not themselves the output of a script: the equations of X_0 (Theorem 6.5(ii)), the identification of Bolza's curve and of $y^2 = x^5 - 1$ (Remark 6.4) from their automorphism groups, and the identification of the classes with N = 18 with the polygons of Fricke and Klein (Remark 6.7). Section 5 says which computed facts they use.

---

2. Installation

You need Python (version 3.10 or later) with a few packages, and GAP (version 4.12 or later). No Sage is needed.

Windows users. The programs run in Linux, which Windows can run inside a window through the Windows Subsystem for Linux (WSL). Install it before the steps below (this is done only once):

1. Press the Windows key, type `PowerShell` (the command line window of Windows; "Terminal" or "Windows Terminal" works as well), right-click on the result and choose **Run as administrator**. Accept the confirmation that Windows shows.
2. In the window that opens, type `wsl --install` and press Enter. This installs Linux (the Ubuntu distribution) and takes a few minutes.
3. Restart the computer.
4. Press the Windows key, type `Ubuntu` and open that application. The first time, it asks you to choose a user name and a password for Linux (they are independent of your Windows account; the password is needed for the `sudo` commands below, and nothing is shown on the screen while you type it).

The Ubuntu window is a Linux terminal: from now on every command of this guide is typed there (PowerShell is not needed any more), following the instructions for Debian/Ubuntu below. Copy this folder into the Linux file system, which is much faster than working on the Windows disk. The Windows disk `C:` is seen from Linux as `/mnt/c/`; for instance, if the folder `girondo_reyes_lowgenus_packings` is on the Windows desktop, type

```
cd
cp -r "/mnt/c/Users/Ana/Desktop/girondo_reyes_lowgenus_packings" .
cd girondo_reyes_lowgenus_packings
```

These are three separate commands, each one followed by Enter. The first one (`cd` alone) takes you to your Linux home folder; in the second one, `Ana` stands for your Windows user name and the final dot, separated from the path by a space, means "here", that is, the destination of the copy (if it is missing, `cp` answers `missing destination file operand`). If you do not know the exact user name or path, list the folders step by step,

```
ls /mnt/c/Users/
ls "/mnt/c/Users/Ana/Desktop/"
```

or type the beginning of a name and press the Tab key, which completes it. Keep the quotation marks if the path contains spaces (as in `OneDrive - Company`). After the third command, `ls` should list the files of this folder (`VERIFYING.md`, `README.md`, the programs `*.py`, ...). To find the folder again in a later session, open Ubuntu and type `cd girondo_reyes_lowgenus_packings`.

GAP. On Debian or Ubuntu (and in WSL) type

```
sudo apt update && sudo apt install gap python3-venv
```

which also installs `python3-venv`, the module that creates Python environments (it is not always present). `sudo` asks for your Linux password (nothing is shown while you type it); if asked `Do you want to continue? [Y/n]`, press Enter. On macOS with Homebrew: `brew install gap`. Otherwise follow the instructions at https://www.gap-system.org. Check that it works:

```
echo 'Print(GAPInfo.Version, "\n"); QUIT;' | gap -q
```

which should print the version number, for instance `4.12.1`. The programs were tested with GAP 4.12.1; an older version (4.11 or before) has not been tested.

Python. Open a terminal **in this folder** (the one that contains this file; in WSL, `cd girondo_reyes_lowgenus_packings`) and type, as two separate commands:

```
python3 -m venv venv
venv/bin/pip install mpmath cypari2 numpy matplotlib openpyxl
```

This creates a private Python environment in the subfolder `venv` (nothing else on your computer is modified) and installs `mpmath` (high precision numerics), `cypari2` (the PARI/GP library, used for exact computations in number fields), `numpy` and `matplotlib`, which are only needed for the figures (Sections 4.4 to 4.6), and `openpyxl`, which is only needed to write the spreadsheet `surfaces_summary.xlsx`; these three can be left out otherwise.

From now on, all the commands are typed in this folder, and `venv/bin/python` is the Python of that environment.

First test.

```
venv/bin/python verify_packings.py fuchs_N9 21
```

After a few seconds the last line must be

```
fuchs_N9     1 surfaces: field OK, side pairings in K and generating K, 4 packings verified
```

(The message `PARI stack size set to ...` printed at the beginning of every program is normal.)

Tested configuration. Linux, Python 3.13, cypari2 2.2.4 (PARI 2.17.2), mpmath 1.4.1, GAP 4.12.1. The programs use nothing specific to Linux and should work in the same way on macOS (where the `python3` provided by the system may be too old: check `python3 --version`, and if it is below 3.10 install a recent one, e.g. `brew install python`) and on Windows through WSL (see above; tested with Windows 11, Ubuntu 26.04 in WSL and GAP 4.15.1, with the same results, the regeneration of Section 6 included). Each program needs about 2 GB of memory (`draw_all.py`, Section 4.6, runs two of them at the same time).

---

3. Verifying all the packings at once (about one minute)

Before running anything else, make a copy of the distributed data. The programs write their results into the folder `groups/`, and you will want to compare them with the original files:

```
cp -r groups groups_distributed
```

Then run

```
venv/bin/python verify_packings.py
```

The output must end with these lines:

```
nec_N12     22 surfaces: field OK, side pairings in K and generating K, 42 packings verified
nec_N9       8 surfaces: field OK, side pairings in K and generating K, 14 packings verified
nec_N8      46 surfaces: field OK, side pairings in K and generating K, 76 packings verified
nec_N7      24 surfaces: field OK, side pairings in K and generating K, 42 packings verified
fuchs_N18    9 surfaces: field OK, side pairings in K and generating K, 19 packings verified
fuchs_N12   25 surfaces: field OK, side pairings in K and generating K, 44 packings verified
fuchs_N10   20 surfaces: field OK, side pairings in K and generating K, 34 packings verified
fuchs_N9    37 surfaces: field OK, side pairings in K and generating K, 70 packings verified
fuchs_N8    77 surfaces: field OK, side pairings in K and generating K, 105 packings verified
fuchs_N7   155 surfaces: field OK, side pairings in K and generating K, 200 packings verified
nec_N18    296 surfaces: field OK, side pairings in K and generating K, 359 packings verified
packings verified: 1005
```

For the NEC families the count includes the canonical double covers (11 + 11 for N = 12, and so on). The first ten lines are the 423 surfaces of the paper (646 packings); the last line is an external test described in Section 7. If any check fails, the program stops with a message saying which surface and which check.

What is checked, for every surface D/K:

1. the matrices of the reflections a, b, c (in `field.py`) have exactly the entries given in Section 2 of the paper and satisfy the relations $a^2 = b^2 = c^2 = (ab)^2 = (bc)^3 = (ca)^N = 1$ exactly, (ca) having order exactly N;
2. every side pairing lies in K, and together they generate K (GAP computes the index of the subgroup they generate);
3. for every extremal packing listed, the stored matrix α is invertible and satisfies $s \cdot \alpha = \pm \alpha \cdot w_s$ exactly, for all side pairings s; that is, $\alpha^{-1} K \alpha$ is contained in the triangle group, so the points $\alpha(g \cdot O)$, g in the triangle group, are the centres of an extremal packing of D/K;
4. the packings listed are pairwise different (for $i \neq j$, $\alpha_i^{-1}\alpha_j$ is not in the triangle group); and their number is the one stated.

To understand what the script does, read its first 30 lines; the whole check is about 200 lines long.

---

4. Checking what the paper says about one surface

4.1 Finding the surface

The surfaces are named after their family and their position in the list computed by GAP:

- `fuchs_N9#21` is the 21st conjugacy class of torsion-free subgroups of index 36 of the triangle group Δ(2,3,9): an orientable 4-extremal surface of genus 2;
- `nec_N8#5` is the 5th class of NEC subgroups of index 48 of Δ±(2,3,8): a non-orientable 3-extremal surface of genus 3; its canonical double cover is called `nec_N8#5+`.

family	**surfaces**	**k**	**N**	**genus**
`nec_N12, nec_N9, nec_N8, nec_N7`	11, 4, 23, 12	1, 2, 3, 6	12, 9, 8, 7	3, non-orientable
`fuchs_N18, fuchs_N12, fuchs_N10, fuchs_N9, fuchs_N8, fuchs_N7`	9, 25, 20, 37, 77, 155	1, 2, 3, 4, 6, 12	18, 12, 10, 9, 8, 7	2, orientable

This is the numbering used in Section 6 and in Appendix A of the paper. To find out which of these surfaces is a surface given by its monodromy (for instance a dessin), see Section 4.7. The surfaces that the paper calls by name are:

in the paper	**name here**
X_0 (Theorem 6.5), also X_6 of Girondo–González-Diez	`fuchs_N18#9` (as a 1-extremal surface) = `fuchs_N9#36` = `fuchs_N9#37` (as a 4-extremal surface)
X_2 of Girondo–González-Diez	`fuchs_N18#2 = fuchs_N9#29`
X_3 (Remark 6.6)	`fuchs_N18#3 = fuchs_N9#21`
X_1, X_4, X_5, X_7, X_8, mirror image of X_8 (Remark 6.7)	`fuchs_N18#1, #4, #8, #5, #6, #7`
Bolza's curve $y^2 = x^5 - x$ (Remark 6.4)	`fuchs_N8#77`
$y^2 = x^5 - 1$ (Remark 6.4)	`fuchs_N10#18`

The spreadsheet `groups/surfaces_summary.xlsx` has one row per surface, with one column for each of: the surface, N, k, orientability, the surface it covers (for the double covers), the index of K in Δ±, the number of extremal packings, the order of the group of isometries induced by the triangle group, whether this group contains an anticonformal isometry, a disc centre of each packing other than the one of the uniformization (in the disc model, one column per packing), the fingerprint `class_id` described below, generators of the surface group K, and the side pairings (written with powers: `(ca)^3cb(ac)^2` means `cacacacbacac`). The same table is in `groups/surfaces_summary.csv` for programs; a spreadsheet set to Spanish or another language may show each line of the `.csv` in a single column, so open the `.xlsx` instead. The columns are described in detail in `README.md` (section Data).

Note on the numbering: GAP 4.12.1 and GAP 4.15.1 produce exactly this order. Another version of GAP might list the classes in another order. For this reason every record of the results files `groups/<family>_results.json` contains a canonical form of the permutation representation (field `key`), and the column `class_id` of the table is a short fingerprint of it. It does not depend on the order of GAP; it is shared by a class and its mirror image (which give mirror-image surfaces) and by a double cover and its orientable class.

4.2 Checking that the packings exist

Take for instance the surface X_3, of which the paper says (Theorem 6.3(iii) and Remark 6.6) that it has four extremal 4-packings, which form two pairs, the packings of each pair sharing a disc:

```
venv/bin/python verify_packings.py fuchs_N9 21
```

```
fuchs_N9#21: 4 packings verified
   packing 0: centre ['-0.16283952106099292529', '-0.44739790710988345808'] (disc model), d
(O, centre) = 1.0358861383423735904
   packing 1: centre ['-2.8166321052040970383e-38', '1.0216698947576522286e-38'] (disc mode
l), d(O, centre) = 0.0
   packing 2: centre ['0.2494846204578588415', '0.043990869858531581801'] (disc model), d(O,
centre) = 0.51794306917118679519
   packing 3: centre ['0.2494846204578588415', '0.043990869858531581801'] (disc model), d(O,
centre) = 0.51794306917118679519
fuchs_N9     1 surfaces: field OK, side pairings in K and generating K, 4 packings verified
```

The four packings exist and are different (Section 3, points 3 and 4). For each one a centre of one of its discs is shown, in the disc model where the centre O of the fundamental polygon is 0: packing 1 is the obvious one, centred at O (the value 10^{-38} is a numerical zero), and packings 2 and 3 have a disc with the same centre, at distance 0.5179 = r_c/2 from O, as stated in Remark 6.6. Packings 0 and 1 also share a disc, which is not visible in this list because only one disc of each packing is shown; all the discs, and the shared ones, are listed in the file produced by `draw_surface.py fuchs_N9 21` (Section 4.4).

4.3 Checking that there are no other packings

Run the search again for this surface:

```
venv/bin/python check_surface.py fuchs_N9 21
```

(for another surface, change the family and the number; `--cover` after them runs the canonical double cover of a non-orientable surface). After a few seconds:

```
[fuchs_N9#21] N=9 k=4 orientable=True  sidepairings=14  D=5.81  #dist=15
   ED-cycle intersections in the circumdisc: 31
   candidates passing the displacement test: 5
   extremal packings found (verified exactly): 4
      centre in disc: (-0.1628395211 - 0.4473979071j)  d(0,z)= 1.035886138
      centre in disc: (-2.816632105e-38 + 1.021669895e-38j)  d(0,z)= 0.0
      centre in disc: (0.2494846205 + 0.04399086986j)  d(0,z)= 0.5179430692
      centre in disc: (0.2494846205 + 0.04399086986j)  d(0,z)= 0.5179430692
   every candidate has a conjugator (several candidates may give the same packing)
```

These are the steps of Section 4 of the paper: the 15 admissible distances up to the bound D = 5.81, the 31 intersection points of the ED-cycles (equi-displaced cycles) of two side pairings inside the circumscribed disc, the 5 candidates that survive the displacement test, and the 4 packings that are found and verified exactly. For surfaces where some candidate has no conjugator, the last line says how many of them are shown not to be centres by an element of K, a product of two side pairings or a single one (it must be all of them).

In the data file `groups/fuchs_N9_results.json` (entry 21) the field `rejected_confirmed_independently` records that every candidate rejected in the last step is displaced by some element of K other than the identity (a side pairing or a product of two side pairings) by a distance that is not admissible, so it cannot be the centre of an extremal disc. (When that displacement is beyond the list of admissible distances of the search, the program computes a longer complete list to decide it; this happens for 10 candidates, on `fuchs_N7#70` and `#71`.)

Following the search step by step. With the option `--details`,

```
venv/bin/python check_surface.py fuchs_N9 21 --details
```

prints, after the summary above, everything that is needed to follow Steps 2 and 3 by hand: the admissible distances d_0, d_1, … up to D; the two side pairings s_1, s_2 used in Step 3 (as words in a, b, c, written with powers: `(ca)^6cbacacbc` is `cacacacacacacbacacbc`), with their translation lengths and the indices i of the distances d_i whose ED-cycles b_s(d_i) = {z : d(z, s z) = d_i} are used; and then every intersection point, numbered `#1`, `#2`, …, with its position in the disc model, the pairs (i, j) such that it lies on b_{s_1}(d_i) and on b_{s_2}(d_j), and its fate. For X_3 the first lines of the list are

```
Intersection points of an ED-cycle of s_1 and one of s_2 in the search disc: 31, of which 5 p
ass the displacement test (candidates).
#1   -0.16283952 -0.44739791i  (distance 1.03588614 from O, angle -110.0000 deg)
      on b_1(d_i) and b_2(d_j) for (i, j) = (0, 8)
      candidate: centre of a disc of packing 2 (verified exactly)
#2   -0.25372509 -0.42302023i  (distance 1.08076471 from O, angle -120.9551 deg)
      on b_1(d_i) and b_2(d_j) for (i, j) = (0, 8)
      discarded: the side pairing cacb(ac)^4ba moves it by 3.78052674, which is not an admiss
ible distance
```

A point is either discarded (and the line says which side pairing moves it by a non-admissible distance), or a candidate that is the centre of a disc of one or more packings (numbered as in the pictures of Section 4.4: packing 1 is the one of the uniformization), or a candidate without a conjugator, and then the line says which element of K (a product of two side pairings) shows that it is not a centre (for instance on `fuchs_N7#54`).

A picture of the ED-cycles.

```
venv/bin/python draw_surface.py fuchs_N9 21 --edcycles
```

writes `surface_figures/fuchs_N9_21_edcycles.pdf` (and `.png`): the central polygon, its circumscribed disc (dashed) and the search disc (dotted, 0.1 larger), the ED-cycles of s_1 (green) and of s_2 (orange), the intersection points discarded by the displacement test (small grey dots) and the candidates (each with its number `#n`): a candidate that is the centre of a disc of packing n carries the marker of that packing, as in Section 4.4 (`#13: P3,P4` means that the point is the centre of a disc of packings 3 and 4, a disc shared by the two packings), and a candidate that is not a centre is marked with a red cross. The file `surface_figures/fuchs_N9_21_edcycles.txt` contains the same report as `check_surface.py --details`, so every point of the picture can be identified there by its number. This is Figure 2 of the paper, for any surface.

When the side pairings move the points of the search disc by large distances (typically for N = 7 and N = 8, and for surfaces with many polygons), many ED-cycles cross the search disc and the picture can be crowded, with curves and labels on top of each other. The picture is only a help: the text file always gives the complete information, point by point.

4.4 A picture of the surface

```
venv/bin/python draw_surface.py fuchs_N9 21
```

writes `surface_figures/fuchs_N9_21.pdf` (and `.png`), with two panels drawn on the fundamental domain made of the k polygons built in Step 1. Edges with the same number are identified by a side pairing (`n~` marks an orientation reversing one); a point on the boundary of the domain appears at each of its copies.

- Left: the centres of the discs of every extremal packing found, one colour and marker per packing. Packing 1 is the one given by the uniformization (its centres are the centres of the polygons). With the option `--tess` the polygons of the tessellations of the other packings are drawn as well (dashed).

- Right: the isometries of the surface (all of them, induced by the triangle group or hidden). Points fixed by some isometry acting as a rotation are marked according to the order of their stabiliser (number of isometries fixing them in this way, including the identity); the curves fixed by isometries acting as reflections (mirrors) are drawn in blue; for orientable surfaces the six Weierstrass points (fixed points of the hyperelliptic involution) are circled. Isometries without fixed points (for instance glide reflections of a non-orientable surface) are only counted, in the text below the picture.

To look at the files, open the folder `surface_figures` with the file browser of your system; in WSL (Windows) the command `explorer.exe surface_figures` opens it in the Windows Explorer. Next to the picture, the text file `surface_figures/fuchs_N9_21.txt` gives the approximate coordinates (in the disc model of the picture, with the hyperbolic distance from O and the angle) of all the relevant points: the centres and vertices of the polygons P1, ..., Pk of the domain (labelled in the picture), the side pairings (which edge of which polygon goes to which, and the word in a, b, c), the centres of the discs of every packing (copies on the boundary of the domain grouped together, and the discs shared by two different packings), the isometries with their orders and numbers of fixed points, the points fixed by rotations with the order of their stabiliser, and the Weierstrass points. With `--dirichlet` the file `..._dirichlet.txt` gives the vertices of the Dirichlet domains and, for each side, the side it is paired with and the word kappa of the pairing.

The text below the picture gives the number of packings, the order of the isometry group (conformal and anticonformal parts, and how many are induced by the triangle group) and a check: the Euler characteristic of the quotient X/G, computed as (chi(X) + total number of isolated fixed points of the nontrivial elements of G)/|G|, must be an integer (and, for the group of conformal automorphisms, an even integer at most 2). A missing or spurious fixed point would make it fail. The induced isometries are treated exactly through the permutation representation; the hidden ones numerically. Options: `--cover` (canonical double cover of a non-orientable surface), `--no-isometries` (left panel only). For k = 6 and k = 12 the domain reaches close to the boundary circle and the outer labels are small; the PDF can be enlarged.

4.5 Dirichlet domains

```
venv/bin/python draw_surface.py fuchs_N9 21 --dirichlet        # centred at 0, a disc centre
of packing 1
venv/bin/python draw_surface.py fuchs_N9 21 --dirichlet-all    # one domain for each packing
```

writes `surface_figures/fuchs_N9_21_dirichlet.pdf` (and `.png`): the Dirichlet domain of the surface group centred at the centre of a disc of the packing (in green, sides with the same number identified, `n~` orientation reversing), with the polygons of the fundamental domain of Step 1 in grey; each picture is centred at the centre of its domain. The domain is computed by `dirichlet.py`: the orbit of the centre consists of centres of the tessellation and is obtained exactly from the permutation representation; the domain is the intersection of the half planes bounded by the bisectors (straight lines in the Klein model), and the computation uses all the orbit points up to twice the largest distance from the centre to a vertex, so that no side can be missing. Below each picture: the number of sides, the lengths of the vertex cycles and the result of the checks (every side is paired, every vertex cycle has angle sum 2π, the area is 2π|χ|); by Poincaré's theorem the side pairings then generate the surface group. For the other

packings the same computation is made for the conjugate group (its coset action is obtained with GAP) and the result is transported back to the fundamental domain of the surface.

4.6 All the pictures at once (about an hour)

```
venv/bin/python draw_all.py
```

draws the 373 surfaces into `surface_figures/all/` (for each surface `<family>_<n>.pdf` as in Section 4.4, `<family>_<n>_dirichlet.pdf` with the Dirichlet domains centred at a disc of each packing, as in Section 4.5, and `<family>_<n>_edcycles.pdf` with `<family>_<n>_edcycles.txt`, the ED-cycles and candidates of Step 3, as in Section 4.3; the distributed folder already contains them; some ED-cycle pictures with N = 7 or 8 are crowded, and then the text file is the one to read), writes the table `surface_figures/all/index.csv` and compares, surface by surface, the number of packings, the order of the induced group and the order of the full isometry group with the data files (`groups/surfaces_summary.csv, groups/surface_identifications.json`), together with the Euler characteristic checks above and the checks of the Dirichlet domains (which must also have at most 18 sides for the orientable surfaces and at most 12 for the non-orientable ones), and the number of candidates of each ED-cycle picture with the results files (every candidate must be either the centre of a disc of a packing or shown not to be one). The report `surface_figures/all/checks.txt` must start with

```
373 surfaces drawn, 0 problems
```

4.7 Identifying a surface given by its monodromy (a few seconds)

If you have a surface of genus 2 or 3 built from regular N-gons, for instance from another paper, from a dessin d'enfant or from your own computation with GAP, you can find out which surface of the catalogue it is. The program `identify_class.py` takes its **monodromy**, i.e. the permutations of the sheets of the covering D/K → D/Δ±(2,3,N) (the same information as the side pairings of a fundamental domain made of N-gons), in one of two forms:

- **three permutations a, b, c**, the action of the three reflections of Δ±(2,3,N) (this works for orientable and non-orientable surfaces): `python3 identify_class.py N abc "<a>" "<b>" "<c>"`;
- **two permutations x, y**, for orientable surfaces: the monodromy of the Belyi function of type (2,3,N), i.e. of the dessin, with x of order 2 and y of order 3 (in either order), whose product has order N: `python3 identify_class.py N xy "<x>" "<y>"`.

The permutations are written either in cycle notation, `"(1,2)(3,4)…"`, as GAP prints them, or as lists of images, `"[2,1,4,3,…]"`; the points are numbered from 1, and their numbering does not matter. The conventions are those of the paper: a, b, c are the reflections in the sides of the triangle described in `README.md` (Conventions), so that ab, bc and ca have orders 2, 3 and N, and x = ab, y = bc; permutations act on the right and products are read from left to right.

Example 1: the surface of Section 7.1 of C. Reyes' thesis. The thesis prints the permutations of a, b, c for its group K_(9,−,2). Copying them:

```
venv/bin/python identify_class.py 9 abc "(1,2)(3,4)(5,6)(7,8)(9,10)(11,12)(13,14)(15,16)(17,1
8)(19,20)(21,22)(23,24)(25,26)(27,28)(29,30)(31,32)(33,34)(35,36)" "(1,3)(2,4)(5,7)(6,8)(9,1
3)(10,14)(11,15)(12,16)(17,23)(18,24)(19,27)(20,28)(21,29)(22,30)(25,33)(26,34)(31,35)(32,3
6)" "(1,4)(2,5)(3,7)(6,9)(8,11)(10,17)(12,19)(13,15)(14,21)(16,25)(18,31)(20,32)(22,24)(23,2
9)(26,28)(27,33)(30,35)(34,36)"
```

prints

```
N = 9: 36 sheets over the orbifold of Delta^pm(2,3,9)
non-orientable surface of genus 3, Euler characteristic -1, index 36 in Delta^pm(2,3,9), made
of 2 regular 9-gons: a 2-extremal surface
canonical form computed; class_id = 3e695f68f5
It is nec_N9#2.
  packings: 2;  isometry group induced by the triangle group: order 2
  disc centres of the other packings (disc model): 0.44755849+0.01658341i
  isometry group: C2 x C2 [4,2]
```

The thesis numbers its sheets differently from GAP 4.12, but the program does not depend on the numbering: the surface is `nec_N9#2`, with the hidden disc centre found in the thesis.

Example 2: a dessin. The monodromy of the dessin of type (2,3,18) of X_0 (the 18-gon of Figure 1):

```
venv/bin/python identify_class.py 18 xy "(1,2)(3,5)(4,7)(6,10)(8,12)(9,11)(13,16)(14,17)(15,1
8)" "(1,3,6)(2,4,8)(5,9,13)(7,11,15)(10,14,18)(12,16,17)"
```

prints, after the first three lines,

```
It is fuchs_N18#9.
  in the paper: X_0 of Theorem 6.5 (X_6 of Girondo-Gonzalez-Diez), as a 1-extremal surface
  packings: 4;  isometry group induced by the triangle group: order 6, with an anticonformal
isometry
  disc centres of the other packings (disc model): 0.36472212-0.63171723i ; -0.12666666-0.718
36233i ; 0.68545336-0.24948462i
  isometry group: D24 [24,6]  conformal automorphisms: D12 [12,4]
```

What the program checks and prints.

- First, that the permutations define a compact surface: they generate a transitive group (the covering is connected), they satisfy the relations of the triangle group, and no point is fixed by a, b, c, ab, bc, ca (or x, y, xy) or by a power of them other than the identity, i.e. the group K has no elliptic elements and no reflections. Otherwise it says which condition fails (for instance `bc has cycles of length [1] < 3: K contains an elliptic element`) and stops. Then it prints the orientability, the genus, the Euler characteristic, the index of K and the number k of N-gons.
- Then it computes the canonical form of the monodromy (the field `key` of the results files) and its fingerprint `class_id`, the column of `groups/surfaces_summary.csv`. Two

monodromies have the same canonical form exactly when they differ by a renumbering of the sheets, i.e. when the two groups K are conjugate in Δ±(2,3,N).
- Finally it looks for the canonical form among the stored classes and prints the class, its name in the paper if it has one, and its lines of `groups/surfaces_summary.csv` and `groups/automorphism_groups.txt`. If the surface is not in the catalogue (for instance if it has genus 4), it says so.

Mirror images. A Fuchsian group K and its mirror image $\sigma K \sigma^{-1}$ (σ a reflection of the triangle group) are conjugate in Δ±(2,3,N), so they have the same canonical form and the same `class_id`; when the surface has no anticonformal automorphism induced by the triangle group they are two different classes of the GAP list, for instance `fuchs_N18#6` (X_8) and `fuchs_N18#7` (its mirror image). For a monodromy given as x, y the program then says which of the two it is with the conventions of the paper, and that with the opposite convention (left actions, or y^{-1} instead of y) it would be the other one: which one is "the" surface and which its mirror image is a matter of convention. For a monodromy given as a, b, c both classes are listed.

Testing the program. `venv/bin/python identify_class.py --selftest` recomputes the canonical form of every stored class (the 373 classes and the 144 surfaces of the external test `nec_N18`) and identifies every class from its own monodromy after a random renumbering of the points, given as a, b, c and, for the Fuchsian classes, as x, y; it must end with `517 classes checked, 0 problems`. The program uses only plain Python and the files in `groups/` (no PARI, no GAP).

---

5. Where each statement of the paper is checked

Each line gives the program, the file it writes in `groups/` (which you can compare with the copy in `groups_distributed/`, see Section 6) and what to look for. Programs not timed take less than a minute.

statement	**program → output**	**what to look for**
Table 2: numbers of classes	`report.py` → `report.txt` (part 4)	e.g. `N= 9 k= 4  classes in Delta:  37  classes in Delta^pm:  24  with induced anticonformal symmetry: 11`
Theorem 6.1, the table	`report.py` → `report.txt` (part 2)	`N= 7 k=6  surfaces=12  packings: 1(3) 2(9)`, etc.; `m(n)` means that n surfaces have m packings
Theorem 6.1, last two sentences	`isometry_orbits.py` → `isometry_orbits.txt`	for every `nec_` family: `not transitive on packings: none` and `isometric surfaces: none`
Theorem 6.2	`report.py` → `report.txt` (part 3)	`same number of packings: all`; the four exceptions to "twice" (order 2 against 12)
Theorem 6.2, Appendix A	`summary_tables.py` → `double_covers.txt`	the table of Appendix A, line by line
Theorem 6.3, the table	`report.py` → `report.txt` (part 5)	e.g. `N= 9 k= 4  per Delta^pm-class: 1(8) 2(13)  4(1)  5(2)`: the two classes with 5 packings are X_0 (see (i)), counted once in the paper
Theorem 6.3(i)	`isometry_orbits.py` → `isometry_orbits.txt`	the only pair of classes giving isometric surfaces: `('fuchs_N9#36', 'fuchs_N9#37')`

statement	program → output	what to look for
Theorem 6.3(ii)	`identify2.py` → `surface_identifications.txt` (7 minutes)	`with only hidden anticonformal automorphisms= 0` in all families; the numbers 9, 25, 20, 36, 77, 155 are the classes in Δ, minus one for N = 9 because of (i)
Theorem 6.3(iii)	`report.py` → `report.txt` (part 5); `draw_surface.py fuchs_N9 29` (and `21, 37`) → `surface_figures/fuchs_N9_29.txt`	only `4(1)` and `5(2)` above 2; the line `discs shared by two different packings`, which is `none` for every other surface (column `shared_discs` of `surface_figures/all/index.csv`)
Theorem 6.3(iv)	`conformal_aut.py fuchs_N9` → `conformal_aut_fuchs_N9.txt`	`summary (one line per Delta^pm-class): {'D6': 2, 'Z2': 6, 'Z2xZ2': 16}`; the two D6 lines are X_0
Theorem 6.3(v)	`multi_extremal.py` → `multi_extremal.txt`	only `fuchs_N18#2, #3, #9` have 4-packings: 2, 4 and 5 of them
Theorem 6.3(vi)	`isometry_orbits.py` → `isometry_orbits.txt`	the only non-transitive cases are `fuchs_N9#36` and `fuchs_N9#37`, i.e. X_0
Theorem 6.5(i)	`fullaut.py` → `full_aut_4_5_packings.txt`; `cross_check_9_18.py` → `cross_check_9_18.txt`	`
Theorem 6.5(ii)	(proved in the paper)	uses Aut⁺(X_0) ≅ D_6 and the quotient orbifold, from the two lines above
Theorem 6.5(iii)	`weierstrass.py` → `weierstrass_X0.txt`	in the block `Frame K9 (N=18, extremal disc):` distances `0.92753857` (angles 60°, 180°, −60°) and `1.4454816` (angles 0°, ±120°); each point appears once or twice, from the two triangles that contain it
Remark 6.4	`groups/surfaces_summary.csv`	`fuchs_N8#77` and `fuchs_N10#18`: one packing, induced isometry groups of order 96 and 20
Remark 6.6 (X_3)	`fullaut.py` → `full_aut_4_5_packings.txt`; `autfix.py groups/fuchs_N9.jsonl 21` (prints only)	`
Remark 6.7 (k = 1)	`report.py` → `report.txt` (part 7); `conformal_aut.py fuchs_N18` → `conformal_aut_fuchs_N18.txt`	distances `0.777654, 1.212306, 1.28738` of the second centre; groups Z2, Z2xZ2, D6 surface by surface
Remark 6.8	`conformal_aut_fuchs_N9.txt`	only X_0 has automorphisms of order 3
Remark 6.9	`aut_groups.py` → `automorphism_groups.txt` (15 minutes)	the column after `Aut+:` takes only the values `C2, C2 x C2, D8, D12, C10` (only `fuchs_N10#18`) and `GL(2,3)` (only `fuchs_N8#77`); for `nec_` lines the column after `Isom:` takes only `C2, C2 x C2, D8, D12`; last line `373 surfaces, 0 problems`. GAP writes D8 and D12 for the dihedral groups of order 8 and 12 (D_4 and D_6 in the paper)
Section 6.2, non-covers	`report.py` → `report.txt` (part 6)	the induced anticonformal automorphisms of the classes that are not double covers

statement	program → output	what to look for
Section 4, "Exact and floating point steps"	`margins.py` → `margins.txt` (8 minutes)	`largest deviation treated as an equality: 1.77e-19; smallest gap that caused a rejection in Step 3: 6.35e-6; candidates rejected in Step 4: 70; smallest gap of their confirmation …: 0.00271`; next line: `confirmed by a product of two side pairings: 70, 10` of them with a longer list of admissible distances
Section 5, Figures 1–3; Section 6, Figures 4 and 5 (Remarks 6.4 and 6.6); Section 7, Figure 6	`figures.py` → folder `figures/`	needs `numpy` and `matplotlib`
Section 7: pictures, coordinates and Dirichlet domains	`draw_surface.py`, `draw_all.py` (Sections 4.4–4.6 above)	`373 surfaces drawn, 0 problems`

To run a program, type `venv/bin/python` followed by its name, for instance `venv/bin/python report.py`.

---

6. Recomputing everything from scratch (about 45 minutes)

This repeats the whole computation, starting from the enumeration of the groups, and then compares every output file with the distributed one. The copy of the distributed data made in Section 3 is needed; the following command makes it only if it does not exist yet:

```
[ -d groups_distributed ] || cp -r groups groups_distributed
```

Then:

```
PATH="$PWD/venv/bin:$PATH" ./regenerate_all.sh
venv/bin/python compare_outputs.py groups_distributed groups
```

The first command empties `groups/` and fills it again, printing the steps as they start (`[1/12]`, `[2/12]`, …; most of them print nothing else while they work, so long silences are normal, and the computer must not go to sleep, which would pause the computation); the second one compares the two folders file by file, ignoring only the running times, and must end with

```
51 files compared: 51 identical, 0 different, 0 missing
```

`regenerate_all.sh` is a short list of commands; you can also run them one by one.

---

7. Additional tests

The paper reports two tests on surfaces outside its scope, which you can repeat with the same programs:

- the eight 1-extremal surfaces of genus 2 (`fuchs_N18`) against Girondo and González-Diez, Israel J. Math. 132 (2002): see Remark 6.7 and Section 5 above;
- the 144 non-orientable extremal surfaces of genus 4 (`nec_N18`, the last line of Section 3) against Nakamura, Conform. Geom. Dyn. 13 (2009): 22 of them have two extremal discs and 122 one. Check with

  ```
  venv/bin/python -c "import json; from collections import Counter; r=json.load(open('gro
  ups/nec_N18_results.json')); print(Counter(x['n_packings'] for x in r if not x['orienta
  ble']))"
  ```

  which prints `Counter({1: 122, 2: 22})`.

---

8. If something goes wrong

- **`gap: command not found`**: GAP is not installed or not on the PATH (Section 2).
- **`No such file or directory: 'groups/...'`**: the programs must be run from this folder.
- **`ModuleNotFoundError: No module named 'cypari2'`**: you are using the system Python instead of `venv/bin/python`.
- **memory errors from PARI**: each program reserves 1 GB for PARI (`pari.allocatemem` in `field.py`); close other programs or lower this value.
- **different class numbers**: see the note on GAP versions in Section 4.1.

The conventions (matrices of a, b, c, coset actions, disc model) are explained in Section 2 of the paper and in `README.md`, which also describes every file.

---

9. Reading the code: where each step of the paper is implemented

The algorithm of Section 4 of the paper is implemented in `surface.py` (class `Surface`), using `field.py` (exact arithmetic) and `geom.py` (floating point geometry):

in the paper	where
matrices of a, b, c and the field L_N (Section 3.3)	`field.py`, class `Field`
torsion-freeness of K, orientability	`Surface.check_torsion_free`
Step 1: fundamental domain and side pairings	`Surface.build_domain`
Step 2: admissible distances (Lemma 3.1, complete list up to D)	`Geometry.centres` in `geom.py`; the bound D in `Surface.analyse`

in the paper	**where**
Step 3: ED-cycles and their intersections in the circumscribed disc (Lemma 3.2), displacement test	`Isom.ed_cycle`, `circle_intersections` and the first part of `Surface.analyse`
Step 4: search of α, reduction algorithm	`Surface.find_alphas`; `Geometry.reduce` in `geom.py`
Step 5: exact verification (Proposition 4.1)	`Surface.certify`
Step 6: counting packings	`Surface.packing_key`, `Surface.same_key`
Section 4.1: induced isometries, conjugacy of the groups K_j, hidden isometries	`batch.py` (`automorphisms`, `canonical_key`), `identify2.py`, `isometry_orbits.py`
margins of the floating point decisions ("Exact and floating point steps")	`margins.py`
independent exact check of all the packings	`verify_packings.py` (it does not use `surface.py` or `geom.py`; from `field.py` it takes the matrices of a, b, c, which it checks against the paper)
isometries, fixed points, Weierstrass points (pictures)	`isometries.py`
Dirichlet domains (Section 7)	`dirichlet.py`
report and picture of Steps 2 and 3 for one surface	`check_surface.details_lines`; `draw_surface.make_edcycles_figure`; the data are recorded by `Surface.analyse` in `S.search`
confirmation that a candidate without α is not a centre	`Surface.confirm_not_centre`
canonical form of a monodromy, identification of a surface in the catalogue	`identify_class.py` (`canonical_key`, `abc_from_xy`, `report`)
isomorphism types of Isom(X) and $\mathrm{Aut}^+(X)$ (Remark 6.9)	`aut_groups.py` (`cayley_table`, `gap_identify`), using `autfix.all_automorphisms`
enumeration of the groups (GAP)	`enumerate.g`, `run_enum.sh`